



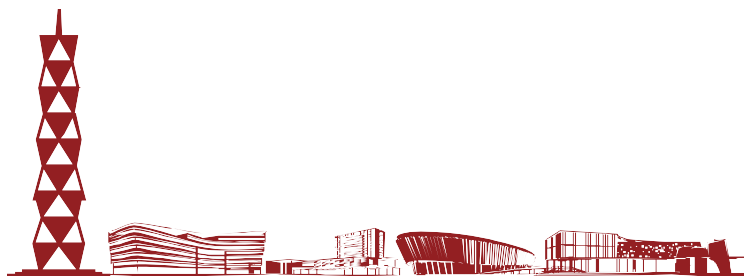
CS240 Algorithm Design and Analysis

Lecture 15

PSPACE: A Class of Problems Beyond NP

Qingying Hao
Spring 2026
2026.04.16

- **Geography.** Amy names capital city c of country she is in. Bob names a capital city c' that starts with the letter on which c ends. Amy and Bob repeat this game until one player is unable to continue. Does Amy have a forced win?
- **Ex.** Budapest $\rightarrow$ Tokyo $\rightarrow$ Ottawa $\rightarrow$ Ankara $\rightarrow$ Amsterdam $\rightarrow$ Moscow $\rightarrow$ Washington $\rightarrow$ Nairobi $\rightarrow$...
- **Geography on graphs.** Given a directed graph $G = (V, E)$ and a start node s , two players alternate turns by following, if possible, an edge out of the current node to an unvisited node. Can first player guarantee to make the last legal move?
- **Remark.** Some problems (especially involving 2-player games and AI) defy classification according to P, NP, and NP-complete



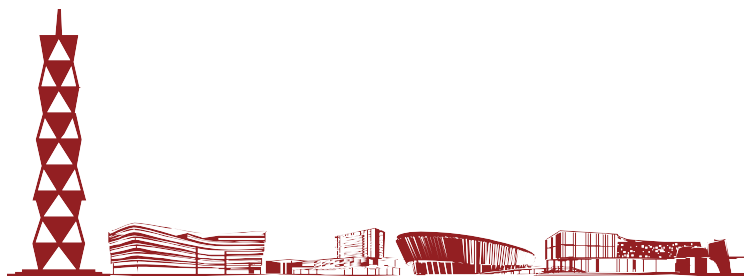


- **P.** Decision problems solvable in polynomial time
- **PSPACE.** Decision problems solvable in polynomial space

- **Observation.** $P \subseteq \text{PSPACE}$

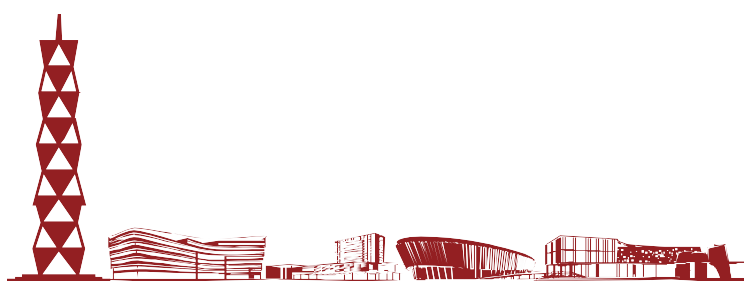


Poly-time algorithm can consume only polynomial space





- **Claim.** 3-SAT is in PSPACE
- **Pf.**
 - Enumerate all 2^n possible truth assignments
 - Check each assignment to see if it satisfies all clauses
- **Theorem.** $NP \subseteq PSPACE$
- **Pf.** Consider arbitrary problem Y in NP
 - Since $Y \leq_p 3\text{-SAT}$, there exists algorithm that solves Y in poly-time plus polynomial number of calls to 3-SAT black box
 - Can implement black box in poly-space





Quantified Satisfiability



- **QSAT (Quantified 3-SAT).** Let $\Phi(X_1, \dots, X_n)$ be a Boolean CNF formula. Is the following propositional formula true?

$$\exists x_1 \forall x_2 \exists x_3 \forall x_4 \dots \forall x_{n-1} \exists x_n \Phi(x_1, \dots, x_n)$$

↑
assume n is odd

- **Intuition.** Amy picks truth value for X_1 , then Bob for X_2 , then Amy for X_3 , and so on. Can Amy satisfy Φ no matter what Bob does?

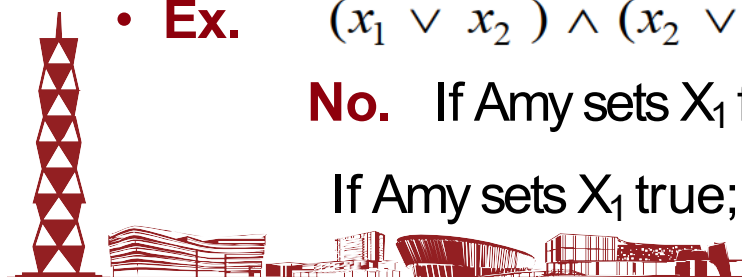
- **Ex.** $(x_1 \vee x_2) \wedge (x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \quad \exists x_1 \forall x_2 \exists x_3 \Phi(x_1, x_2, x_3)$

Yes. Amy sets X_1 true; Bob sets X_2 ; Amy sets X_3 to be same as X_2

- **Ex.** $(x_1 \vee x_2) \wedge (\bar{x}_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$

No. If Amy sets X_1 false; Bob sets X_2 false; Amy loses;

If Amy sets X_1 true; Bob sets X_2 true; Amy loses



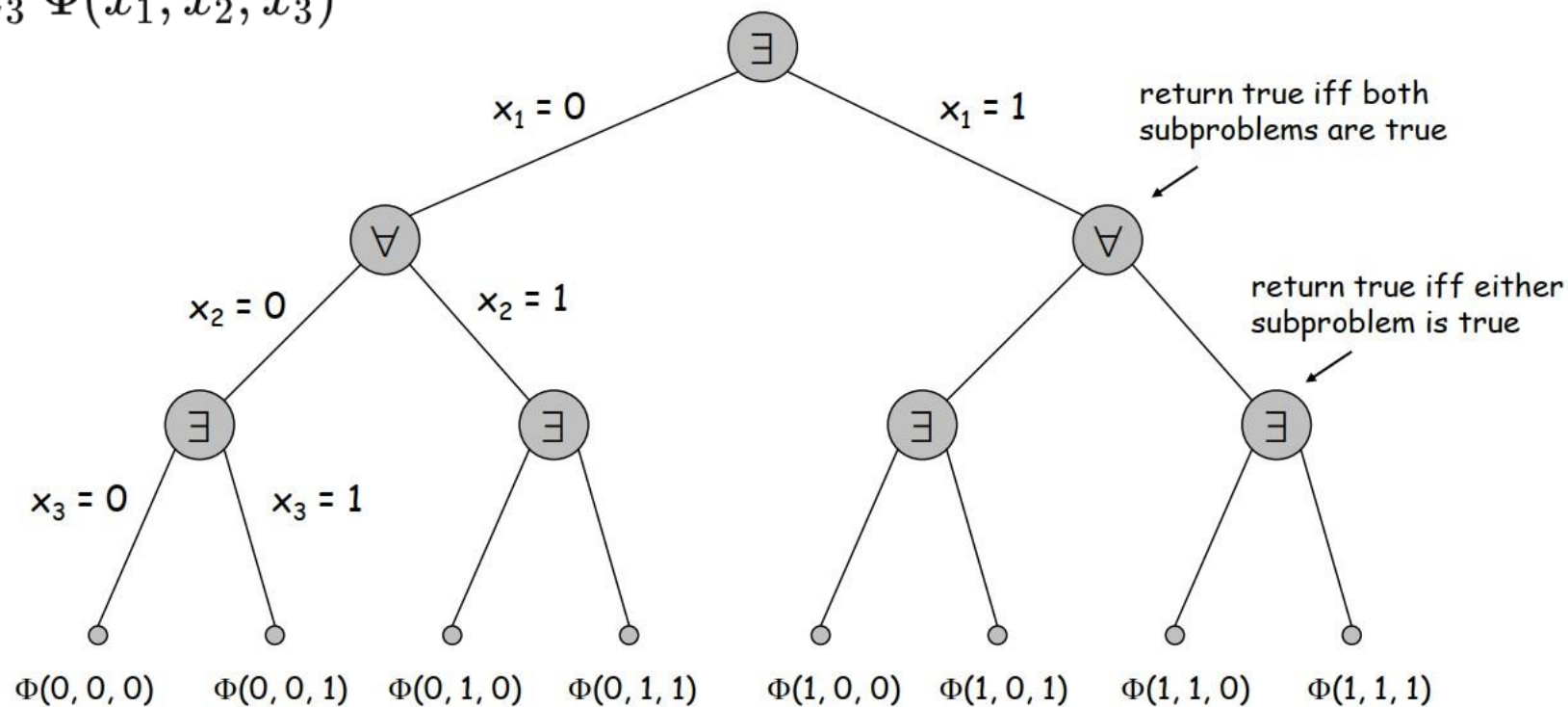


QSAT is in PSPACE



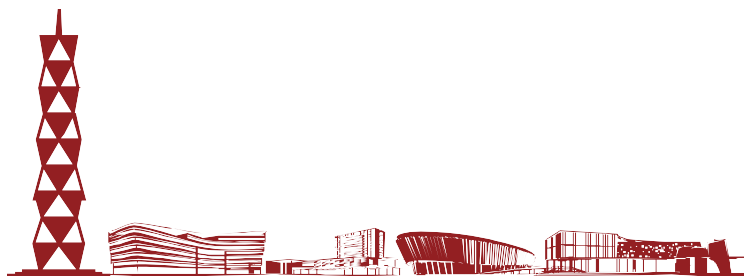
- **Theorem.** QSAT $\in$ PSPACE
- **Pf.** Recursively try all possibilities
 - Only need one bit of information from each subproblem
 - Amount of space is proportional to depth of function call stack

$$\exists x_1 \forall x_2 \exists x_3 \Phi(x_1, x_2, x_3)$$





Planning Problem

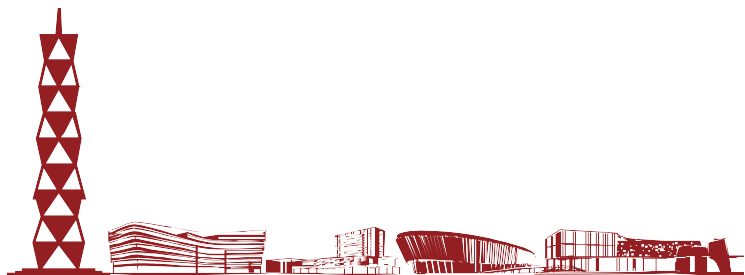
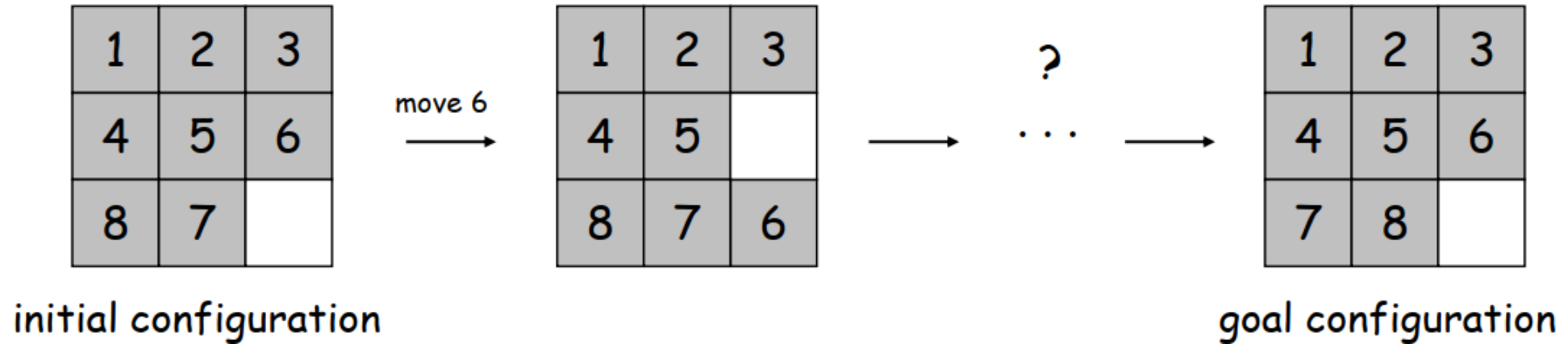




15-Puzzle



- **8-puzzle, 15-puzzle.** [Sam Loyd 1870s]
 - Board: 3-by-3 grid of tiles labeled 1-8
 - Legal move: slide neighboring tile into blank (white) square
 - Find sequence of legal moves to transform initial configuration into goal configuration



- **Conditions.** Set $C = \{C_1, \dots, C_n\}$
- **Initial configuration.** Subset $c_0 \subseteq C$ of conditions initially satisfied
- **Goal configuration.** Subset $c^* \subseteq C$ of conditions we seek to satisfy
- **Operations.** Set $O = \{O_1, \dots, O_k\}$
 - To invoke operator O_i , must satisfy certain prereq conditions
 - After invoking O_i certain conditions become true, and certain conditions become false
- **PLANNING.** Is it possible to apply sequence of operators to get from initial configuration to goal configuration?
- **Examples.**
 - 15-puzzle
 - Rubik's cube
 - Logistical operations to move people, equipment, and materials



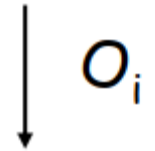


Planning Problem: 8-Puzzle



- **Planning example.** Can we solve the 8-puzzle?
- **Conditions.** $C_{ij}, 1 \leq i, j \leq 9.$ $\leftarrow C_{ij}$ means tile i is in square j
- **Initial state.** $c_0 = \{C_{11}, C_{22}, \dots, C_{66}, C_{78}, C_{87}, C_{99}\}.$
- **Goal state.** $c^* = \{C_{11}, C_{22}, \dots, C_{66}, C_{77}, C_{88}, C_{99}\}.$
- **Operators.**
 - Precondition to apply $O_i = \{C_{78}, C_{99}\}$
 - After invoking O_i , conditions C_{79} and C_{98} become true
 - After invoking O_i , conditions C_{78} and C_{99} become false

1	2	3
4	5	6
8	7	9



1	2	3
4	5	6
8	9	7





Planning Problem: In Exponential Time



- **Configuration graph G**
 - Include node for each of 2^n possible configurations
 - Include an edge from configuration c' to configuration c'' if one of the operators can convert from c' to c''
- **PLANNING.** Is there a path from c_0 to c^* in configuration graph?
- **Claim.** PLANNING is in EXPTIME
- **Pf.** Run BFS to find path from c_0 to c^* in configuration graph
- **Note.** Configuration graph can have 2^n nodes, and shortest path can be of length $= 2^n - 1$



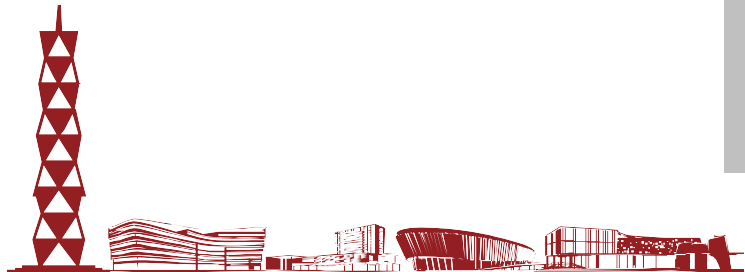


Planning Problem: In Polynomial Space



- **Theorem.** PLANNING is in PSPACE
- Pf.
 - Suppose there is a path from c_1 to c_2 of length $\leq L$
 - Path from c_1 to midpoint and from midpoint to c_2 are each $\leq L/2$
 - Enumerate all possible midpoints
 - Apply recursively. Depth of recursion = $\log_2 L$

```
boolean hasPath( $c_1$ ,  $c_2$ , L) {  
    if (L = 1) return correct answer  
  
    foreach configuration  $c'$  {  
        boolean x = hasPath( $c_1$ ,  $c'$ ,  $\lceil L/2 \rceil$ )  
        boolean y = hasPath( $c'$ ,  $c_2$ ,  $\lceil L/2 \rceil$ )  
        if (x and y) return true  
    }  
    return false  
}
```



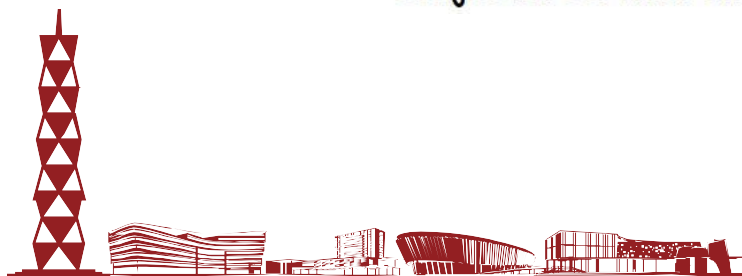


PSPACE-Complete



- **PSPACE** Decision problems solvable in polynomial space
- **PSPACE-Complete.** Problem Y is PSPACE-complete if (i) Y is in PSPACE and (ii) for every problem X in PSPACE, $X \leq_p Y$
- **Theorem.** [Stockmeyer-Meyer 1973] QSAT is PSPACE-complete
- **Theorem.** $\text{PSPACE} \subseteq \text{EXPTIME}$
- **Pf.** Previous algorithm solves QSAT in exponential time, and QSAT is PSPACE-complete
- **Summary.** $P \subseteq NP \subseteq \text{PSPACE} \subseteq \text{EXPTIME}$

↑ ↑ ↑
it is known that $P \neq \text{EXPTIME}$, but unknown which inclusion is strict;
conjectured that all are

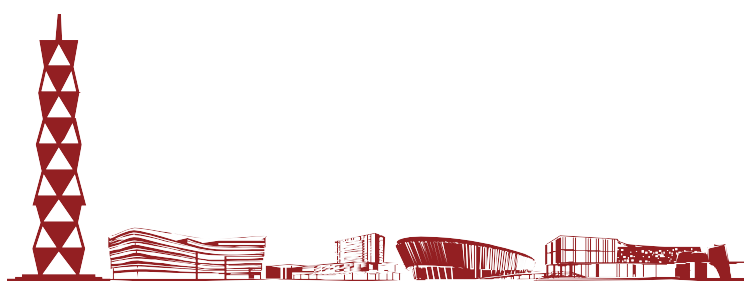




PSPACE-Complete Problems



- More PSPACE-complete problems
 - **Competitive facility location**
 - Natural generalizations of games
 - Othello, Hex, Geography, Rush-Hour, Instant Insanity
 - Shanghai, go-moku, Sokoban
 - Given a memory restricted Turing machine, does it terminate in at most k steps?
 - Do two regular expressions describe different languages?
 - Is it possible to move and rotate complicated object with attachments through an irregularly shaped corridor?
 - Is a deadlock state possible within a system of communicating processors?





Competitive Facility Location

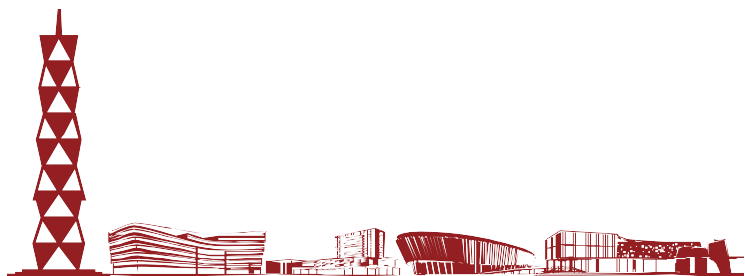


- **Input.** Graph with positive node weights, and target number B
- **Game.** Two competing players alternate in selecting nodes. Not allowed to select a node if any of its neighbors has been selected
- **Competitive facility location.** Can second player guarantee at least B units of profit?

(In general, the graph may not be a chain.)



Yes if $B = 20$; no if $B = 25$.

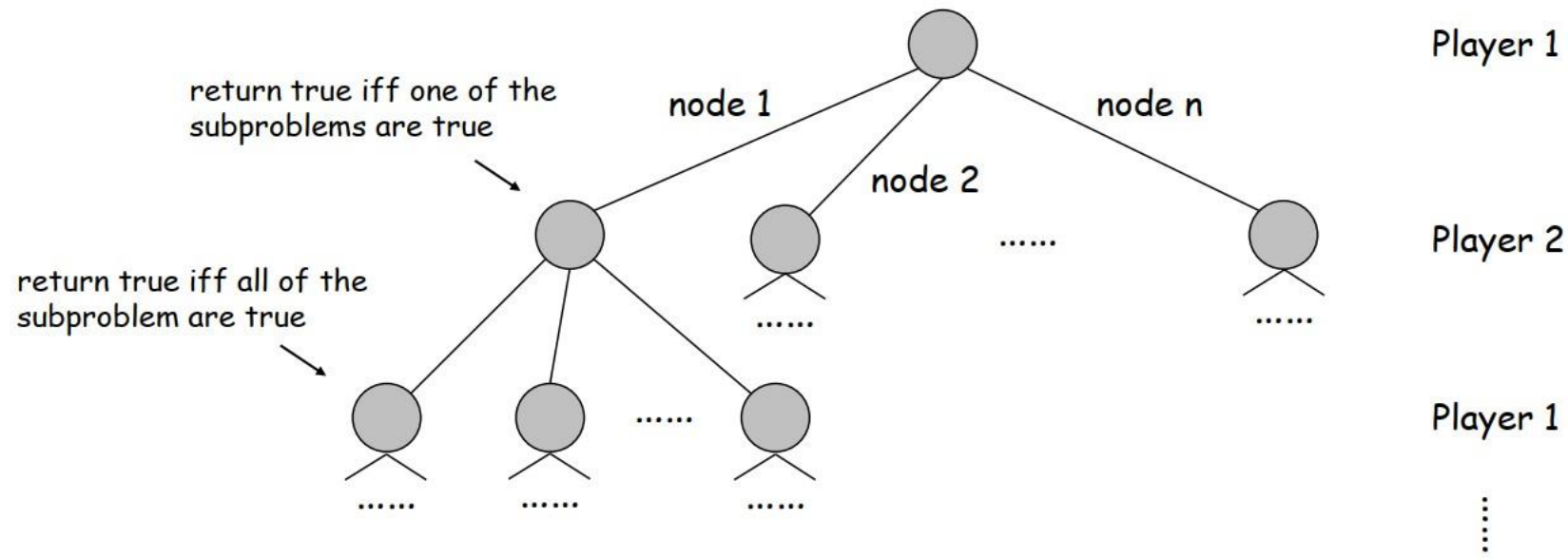




Competitive Facility Location



- **Claim.** COMPETITIVE-FACILITY is PSPACE-complete
- Pf.
- To solve it in poly-space, use recursion like QSAT, but at each step there are up to n choices instead of 2

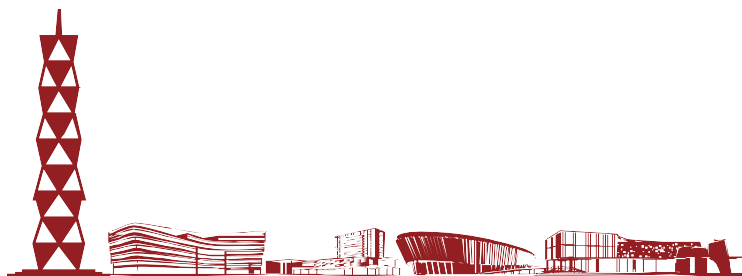




Competitive Facility Location



- **Claim.** COMPETITIVE-FACILITY is PSPACE-complete
- **Pf.**
 - To show that it's complete, we show that QSAT polynomial reduces to it.
 - Given an instance of QSAT, we construct an instance of COMPETITIVE-FACILITY such that player 2 can force a win iff QSAT formula is **false**





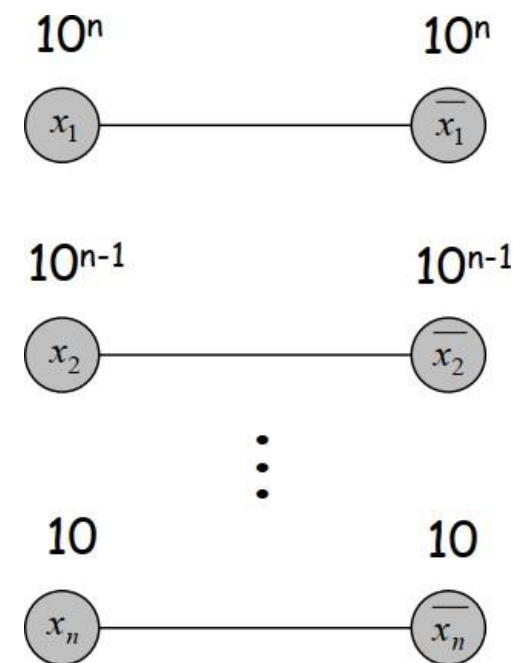
Competitive Facility Location



Assume n is odd



- **Construction.** Given instance $\Phi(X_1, \dots, X_n) = C_1 \wedge C_2 \wedge \dots \wedge C_k$ of QSAT
 - Include a node for each literal and its negation and connect them
 - At most one of X_i and its negation can be chosen
 - Choose a large constant c (e.g., $c \geq k+2$), and put weight c^{n-i+1} on literal x_i and its negation;
 - Set $B = c^{n-1} + c^{n-3} + \dots + c^4 + c^2 + 1$
 - This ensures variables are selected in order $x_1, x_2, \dots, x_n$
 - As is, player 2 will lose by 1 unit: $c^{n-1} + c^{n-3} + \dots + c^4 + c^2$



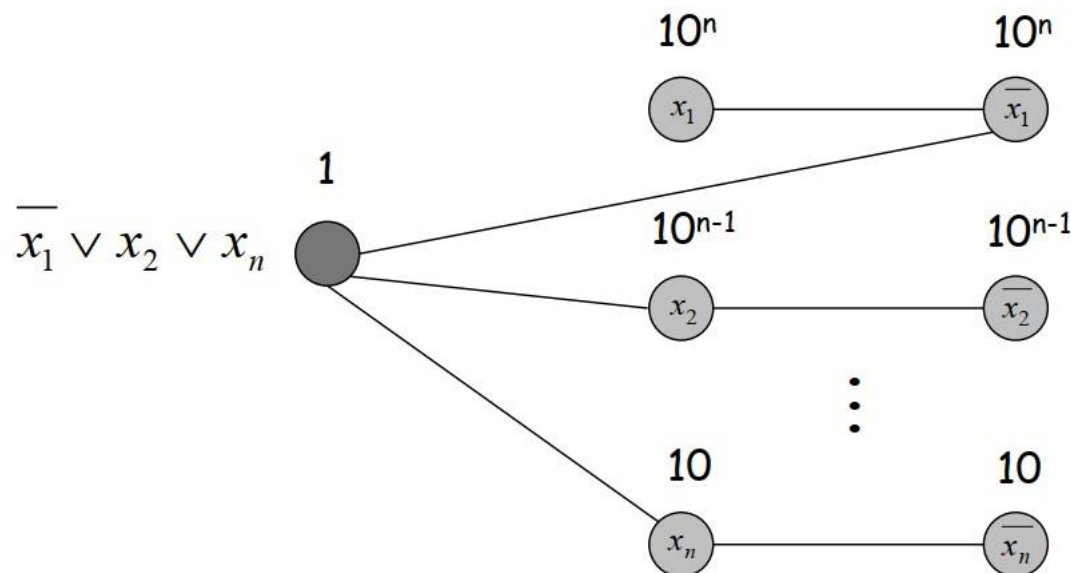


Competitive Facility Location



- **Construction.** Given instance $\Phi(X_1, \dots, X_n) = C_1 \wedge C_2 \wedge \dots \wedge C_k$ of QSAT
 - Given player 2 one last move on which she can try to win
 - For each clause C_j , add node with value 1 and an edge to each of its literals
 - Player 2 can make last move iff truth assignment defined alternately by the players failed to satisfy some clause, i.e.,

$$\begin{aligned} & \forall x_1 \exists x_2 \forall x_3 \exists x_4 \dots \exists x_{n-1} \forall x_n \neg \Phi(x_1, \dots, x_n) \\ \Leftrightarrow & \neg \exists x_1 \forall x_2 \exists x_3 \forall x_4 \dots \forall x_{n-1} \exists x_n \Phi(x_1, \dots, x_n) \end{aligned}$$





Summary





- **PSPACE** Decision problems solvable in polynomial **space**
- **PSPACE problems**
 - QSAT
 - Planning
- **Theorem.** $NP \subseteq PSPACE \subseteq EXPTIME$
- **PSPACE-Complete.** Problem Y is PSPACE-complete if (i) Y is in PSPACE and (ii) for every problem X in PSPACE, $X \leq_p Y$
- **PSPACE-Complete problems**
 - QSAT
 - Competitive Facility Location





Complexity Classes

